

Universidade Federal de Minas Gerais (UFMG)

Engenharia de Controle e Automação

Sistema Automatizado de Inspeção de Túneis

Trabalho Final - Automação em Tempo Real - Etapa 1

Alunos:

Ana Beatriz Soares Cardoso – 2022108170

Julia Pereira Maia Ribeiro – 2021014864

Pedro Eduardo Alvino Tapia – 2021031084

Belo Horizonte

2026

Índice

1	Introdução	1
2	Proposta da Arquitetura	1
2.1	Visão Geral dos Processos	2
2.2	Front-End — Simulação e Interface do Operador	2
2.3	Back-End — Controle Embarcado Concorrente	3
2.3.1	Threads Cíclicas de Tempo Real	3
2.3.2	Threads Reativas (Aperiódicas)	4
2.4	Sincronização, Buffers e Comunicação entre Threads - Memória Compartilhada	5
2.4.1	NavBuffer — Dados de Navegação	5
2.4.2	SensorBuffer - Dados dos Sensores	6
2.5	APIs e Gerenciamento das Tarefas Cíclicas	7
3	Resultados dos Testes Preliminares	8
4	Conclusão	8
4.1	Próximos Passos e Testes	8
5	Apêndice	11
5.1	<i>main.cpp</i>	11
5.2	<i>shared_buffers.hpp</i>	14
5.3	<i>tasks.hpp</i>	15
5.4	<i>tasks.cpp</i>	15
5.5	<i>PIDController.hpp</i>	20
5.6	<i>PIDController.cpp</i>	20
5.7	<i>NavegationManager.hpp</i>	22
5.8	<i>NavegationManager.cpp</i>	22
5.9	<i>Camera.hpp</i>	23
5.10	<i>Camera.cpp</i>	24
5.11	<i>Coletor.hpp</i>	25
5.12	<i>Coletor.cpp</i>	26
5.13	<i>Odometria.hpp</i>	27
5.14	<i>Odometria.cpp</i>	28
5.15	<i>Lidar.hpp</i>	28
5.16	<i>Lidar.cpp</i>	29

1 Introdução

O monitoramento e a manutenção preventiva de infraestruturas críticas subterrâneas, como túneis rodoviários e metroviários, apresentam desafios severos de segurança operacional e eficiência de inspeção. Tradicionalmente, a identificação de anomalias estruturais, tais como falhas na superfície do teto, infiltrações ou deformações geométricas, depende de inspeções visuais manuais intermitentes, as quais expõem os operadores a ambientes de risco e demandam a interrupção prolongada do tráfego.

Para mitigar essas limitações, o presente trabalho aborda o desenvolvimento de um sistema computacional concorrente embarcado para o controle e telemetria de um robô de inspeção autônoma de túneis. O problema central consiste em gerenciar, em tempo real, a concorrência e a sincronização de tarefas com diferentes restrições temporais (deadlines), divididas essencialmente em duas frentes integradas:

- **Controle Dinâmico de Navegação:** A execução de algoritmos de comando e controle em malha fechada (PID) que devem responder deterministicamente a uma frequência fixa para garantir a estabilidade cinemática do veículo.
- **Percepção e Inspeção Reativa:** O processamento assíncrono de dados volumosos provenientes de sensores de varredura (LIDAR) e sistemas de visão artificial (IA por Câmera), que exigem uma redução coordenada na velocidade do robô (slowdown) ao detectarem variações geométricas severas no teto para viabilizar registros fotográficos precisos.

Dessa forma, a arquitetura do software deve ser estruturada para mitigar condições de corrida (race conditions) no acesso aos buffers de memória compartilhada e evitar o desperdício de ciclos de processamento da CPU, utilizando primitivas rigorosas de exclusão mútua (mutexes) e sincronização orientada a eventos (variáveis de condição).

2 Proposta da Arquitetura

A arquitetura do sistema foi concebida segundo o princípio de separação clara entre **simulação/operador** e **controle embarcado concorrente**, interligados por comunicação assíncrona baseada em mensagens. O objetivo é garantir determinismo nas tarefas críticas de controle e, simultaneamente, flexibilidade para tarefas de percepção e inspeção.

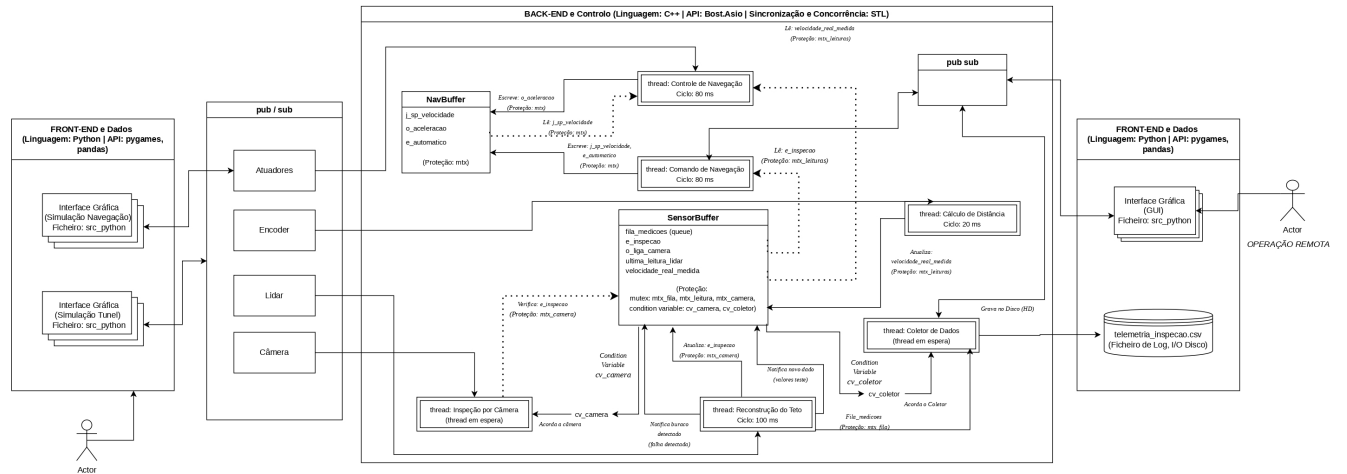


Figura 1: Proposta da Arquitetura - Sistema Completo.

O sistema é dividido em dois processos principais que executam em paralelo: Back-End (C++) e Front-End (Python-pygames).

2.1 Visão Geral dos Processos

Front-End (Python)

Responsável pela simulação, interface com o operador e geração dos dados sensoriais.

Back-End (C++)

Responsável pelo controle embarcado concorrente, processamento de sensores e persistência de dados.

A comunicação entre os processos é realizada por meio de um modelo **Publish/Subscribe**, permitindo desacoplamento entre simulação e controle.

2.2 Front-End — Simulação e Interface do Operador

O processo Front-End será implementado em Python utilizando Pygame e Pandas. Ele irá possuir três responsabilidades principais:

- Simulação da navegação do robô no túnel;
- Renderização dos sensores (LIDAR, câmera e encoders);
- Interação com o operador remoto.

A Interface Gráfica permite o envio de comandos de operação e a visualização em tempo real da telemetria do sistema.

Os dados produzidos pelo Front-End são publicados via Pub/Sub e consumidos pelo processo embarcado.

2.3 Back-End — Controle Embarcado Concorrente

O Back-End é implementado em C++ utilizando as APIs:

- `std::thread`, para concorrência;
- `Boost.Asio`, para timers e comunicação assíncrona.

Estas serão explicadas no tópico *Sincronização, Buffers e Comunicação entre Threads - Memória Compartilhada* deste mesmo relatório.

O processo Back-End é composto por seis threads concorrentes, sendo elas: *Thread de Comando de Navegação*, *Thread de Controle de Navegação*, *Thread do Cálculo da Distância*, *Thread de Reconstrução do Teto*, *Thread de Inspeção por Câmera* e *Thread do Coletor de Dados*. Cada uma delas tem diferentes requisitos temporais, separados em **threads Cíclicas de Tempo Real** e **Threads Reativas**.

2.3.1 Threads Cíclicas de Tempo Real

2.3.1.1 Thread de Comando de Navegação — 80 ms

Responsável pela aquisição das diretrizes de movimento e gerenciamento dos estados de operação do robô, atuando como o elo entre a interface (ou lógica autônoma) e o controle dinâmico.

Avalia periodicamente as entradas do sistema. Seu principal diferencial de concorrência é a leitura segura da flag `e_inspecao` (protegida pelo `mtx_camera`), que dita se o robô deve entrar em modo de Slowdown (redução de velocidade para 1.0 m/s). Após processar a lógica no `NavigationManager`, a thread adentra a zona crítica do `NavBuffer` (através de um `std::lock_guard` no `nav.mtx`) para atualizar o setpoint de velocidade (`j_sp_velocidade`) e o modo de operação (`e_automatico`), garantindo que o controlador PID sempre leia referências íntegras.

Escreve no **NavBuffer**: - setpoint de velocidade (`j_sp_velocidade`) - modo de operação (`e_automatico`)

2.3.1.2 Thread de Controle de Navegação — 80 ms

Consiste na malha fechada de controle PID. Para garantir a estabilidade dinâmica e evitar respostas ruidosas, essa thread exige determinismo estrito na leitura dos dados.

A cada ciclo de 80 ms, ela adquire dois bloqueios de forma hierárquica para evitar contenção: primeiro, lê o setpoint atual no `NavBuffer` (`nav.mtx`); em seguida, lê a variável `velocidade_real_medida` no `SensorBuffer` (`mtx_leituras`). Em posse dos dados, a instância de `PIDController` calcula o esforço compensatório e escreve a saída `o_aceleracao` de volta no `NavBuffer`. Esse isolamento impede que o cálculo matemático sofra interferência das rotinas de odometria que rodam em frequências mais altas (20ms).

Operações principais: - Lê velocidade real medida - Lê setpoint de velocidade - Calcula erro e gera esforço de aceleração (`o_aceleracao`) - Atualiza o `NavBuffer`

2.3.1.3 Thread de Cálculo de Distância — 20 ms

Responsável por atuar como simuladora da física (integração do esforço de aceleração) e Produtora de dados telemétricos.

Devido ao seu deadline restrito de 20 ms, ela interage com a memória compartilhada de forma cirúrgica. Ela atualiza a `velocidade_real_medida` e lê a `ultima_leitura_lidar` bloqueando rapidamente o `mtx_leituras`. Em seguida, empacota esses dados na estrutura `Medicao` e adentra a zona crítica da fila de medições (`mtx_fila`) apenas pelo tempo necessário para realizar um `.push()`. Ao final do ciclo, dispara a notificação `cv_coletor.notify_one()`, avisando de forma não-bloqueante a thread de gravação que um novo pacote está disponível.

Responsável por: - Processar odometria - Atualizar velocidade real medida

Os dados são armazenados no `SensorBuffer` com proteção por mutex.

2.3.1.4 Thread de Reconstrução do Teto — 100 ms

Responsável pela percepção ambiental contínua e detecção de anomalias estruturais. Utiliza uma instância de `LidarFilter` para aplicar uma média móvel circular, mitigando falsos positivos e ruídos do sensor.

Quando a média processada ultrapassa as margens de tolerância (configurando buracos ou saliências), esta thread atua como o gatilho reativo do sistema. Ela tranca o `mtx_camera`, altera as variáveis de estado `e_inspecao` (sinalizando a frenagem para a navegação) e o `liga_camera`. Imediatamente a seguir, dispara a variável de condição `cv_camera.notify_one()`, acordando a thread de visão computacional.

Ao identificar falhas severas: - Atualiza a flag `e_inspecao` - Dispara a variável de condição `cv_camera`

2.3.2 Threads Reativas (Aperiódicas)

2.3.2.1 Thread de Inspeção por Câmera

Modelada para absorver processamento computacional intensivo (simulando inferência de IA) sem comprometer o scheduler de tempo real.

Opera suspensa no núcleo do sistema operacional (`cv_camera.wait()`), consumindo 0% de CPU. Ao ser notificada pelo LIDAR de que uma anomalia foi encontrada, a thread acorda. O grande mérito arquitetural desta tarefa é o desbloqueio explícito do mutex (`lock.unlock()`) antes de iniciar o processamento pesado `pesado_ia()`. Essa técnica garante que as threads de navegação continuem operando livremente para ler o estado de Slowdown enquanto a “IA” trabalha. Concluída a simulação, a thread retoma o bloqueio apenas para resetar as flags e liberar o robô para a velocidade de cruzeiro.

Fluxo: 1. Aguarda em `cv_camera.wait()` 2. Ao ser acordada, executa algoritmos de visão computacional 3. Gera dados estruturados de inspeção

2.3.2.2 Thread Coletora de Dados

Atua como Consumidora no padrão Produtor-Consumidor. O gargalo clássico de sistemas de tempo real é o acesso ao disco (I/O), que possui latência imprevisível.

Para isolar o sistema desse gargalo, o Coletor aguarda passivamente os sinais da Odometria via `cv_coletor`. Quando acorda, retira as mensagens da fila `_medicoes` (protegida por `mtx_fila`), processa o cálculo dinâmico de nível de confiança e — crucialmente — destranca o mutex da fila antes de chamar o método `logger.gravar(dado)`. Isso assegura que o salvamento lento no arquivo `.csv` jamais impeça a thread de Odometria (20 ms) de depositar novos dados na memória, mitigando completamente o risco de buffer overflow ou estouro de deadline.

Fluxo: 1. Aguarda em `cv_coletor.wait()` 2. Consome dados da fila compartilhada 3. Grava no arquivo `telemetria_inspecao.csv`

Essa separação evita bloqueios nas threads críticas de controle.

2.4 Sincronização, Buffers e Comunicação entre Threads - Memória Compartilhada

A comunicação entre as seis tarefas do sistema embarcado foi projetada utilizando o modelo de **Memória Compartilhada**. Para garantir a integridade dos dados e o determinismo temporal exigido em sistemas de tempo real, a arquitetura foi segmentada em buffers especializados com *granularidade fina*. Essa abordagem evita o uso de um único mutex global para toda a memória, o que causaria severo bloqueio mútuo (contention) e degradação do desempenho das threads críticas.

A Sincronização e a Exclusão Mútua foram implementadas utilizando as primitivas `std::mutex` e `std::lock_guard` da biblioteca padrão do C++17, operando sob três pilares fundamentais:

- **Garantia de Exclusão Mútua (Zonas Críticas):** Proteção de variáveis acessadas por múltiplas threads simultaneamente (como a *velocidade real medida* e o *setpoint*), mitigando completamente as condições de corrida (*race conditions*).
- **Sincronização Orientada a Eventos:** Uso de variáveis de condição (`std::condition_variable`) para suspender threads reativas (Câmera e Coletor), garantindo que elas consumam 0% de processamento da CPU enquanto aguardam os sinais de ativação (*notify_one*).
- **Isolamento de Chamadas de Sistema (E/S):** Uso do mutex `mtx_console` para serializar o acesso ao fluxo `std::cout`, impedindo o atropelamento de strings no terminal e garantindo logs perfeitamente cronológicos.

2.4.1 NavBuffer — Dados de Navegação

```
1 struct NavBuffer {
2     double j_sp_velocidade = 0.0; // Velocidade alvo solicitada (m/s)
3     double o_aceleracao = 0.0;      // Esforço do controle calculado pelo PID
4     bool e_automatico = false; // Estado de operação (Manual ou Auto)
5     std::mutex mtx;                // O cadeado do PID
6 };
```

O *NavBuffer* funciona como a ponte de controle entre a camada de decisão (o operador ou a lógica automática de estados) e a camada de atuação física (o algoritmo PID). Ele foi projetado para isolar as variáveis de referência de movimento do veículo.

- **j_sp_velocidade (double)**: Armazena o Setpoint ativo do robô em (m/s). A transição dessa variável para o tipo double foi necessária para permitir que os estados de desaceleração escalonada (como os 0.2 m/s da inspeção) fossem transmitidos sem truncamento matemático para a malha do PID.
- **o_aceleracao (double)**: Guarda o esforço de controle (a aceleração calculada em m/s^2) gerado pelo PID. A thread de odometria lê esse valor continuamente para simular a resposta física do motor no ambiente por meio de integração numérica.
- **e_automatico (bool)**: Uma flag de estado que dita se o robô deve seguir as ordens dinâmicas do joystick manual ou a velocidade nominal do modo automático.
- **std::mutex mtx**: O cadeado exclusivo deste buffer. Ele garante que a thread de Comando de Navegação possa atualizar o Setpoint (escrevendo no buffer) ao mesmo tempo em que a thread do PID faz a leitura desses alvos a cada 80 ms, sem que ocorra leitura de dados parciais ou corrompidos.

2.4.2 SensorBuffer - Dados dos Sensores

```
1 struct SensorBuffer {
2     // ----- Fila de Dados (Produtor-Consumidor) -----
3     std::queue<Medicao> fila_medicoes;
4     std::mutex mtx_fila; // Cadeado para a inserção e remoção da fila
5     std::mutex mtx_leituras;
6
7     std::condition_variable cv_coletor; // Variável de condição para o Coletor de Dados.
8
9     // ----- Sistema de Alarme - Interrupção (Câmera) -----
10    bool e_inspecao = false; // FLAG que indica se o teto tem buracos
11    std::mutex mtx_camera; // Cadeado para proteger a alteração da FLAG
12
13    bool o_liga_camera = false; // Variável de Condição para a Câmera
14    std::condition_variable cv_camera; // O alarme da câmera
15
16    float ultima_leitura_lidar = 10.0;
17    double velocidade_real_medida = 0.0; // Variável para compartilhar com o PID
18 };
```

O *SensorBuffer* é a estrutura de dados mais complexa do sistema, atuando na telemetria e detecção de falhas. Ele gerencia três fluxos concorrentes distintos:

- **A Fila de Mensagens (*fila_medicoes*)**: Implementa um padrão de arquitetura Produtor-Consumidor assíncrono. A thread de Cálculo de Distância atua como a *Produtora*, empacotando os dados do Encoder e do LIDAR na estrutura Medicao e inserindo-os na fila (*fila_medicoes.push()*). A thread Coletor de Dados atua como a *Consumidora*, retirando os pacotes e

salvando-os no HD. Esse arranjo é protegido pelo *mtx_fila* e sincronizado pela variável de condição *cv_coletor*, garantindo que a Coletora só acorde quando houver dados prontos para escrita.

- **A Malha de Interrupção da Câmera (*e_inspecao*, *o_liga_camera* e *cv_camera*):** Funciona como um sistema reativo de alarme por software. Quando a thread do LIDAR detecta uma anomalia na superfície, ela tranca o *mtx_camera*, ativa as flags *e_inspecao* sinalizando o freio imediato para a Navegação, e *o_liga_camera*, por fim disparando o sinal *cv_camera.notify_one()*. Isso faz a thread Inspeção Câmera acordar instantaneamente para iniciar o processamento pesado de registro da superfície.
- **Compartilhamento de Variáveis Rápidas (*ultima_leitura_lidar* e *velocidade_real_medida*):** Variáveis globais protegidas pelo *mtx_leituras*. Elas servem para troca rápida de dados de telemetria entre threads cíclicas (como o PID lendo a velocidade real instantânea calculada pela Odometria), evitando que o PID precise buscar dados dentro da fila de medições e perca o seu deadline.

2.5 APIs e Gerenciamento das Tarefas Cíclicas

Para gerenciar a execução periódica das tarefas de tempo real sem ocorrer acúmulo de atrasos (*drift temporal*) e flutuações (*jitter*), causados por funções de suspensão simples como o *sleep()*, foi utilizada a biblioteca *Boost.Asio* através da classe *boost::asio::steady_timer*.

O mecanismo baseia-se em um modelo assíncrono não-bloqueante operado por loops baseados em chamadas recursivas indiretas via funções lambda capturadas por referência ([&]). Ao final de cada ciclo de execução de uma tarefa, o timer correspondente tem seu próximo instante de disparo incrementado deterministicamente a partir do seu ponto de expiração prévio, e não a partir do tempo atual do sistema. Isso assegura que o período médio da tarefa mantenha-se estrito e imune ao tempo de execução interno do código de cada thread.

O gerenciamento das tarefas foi distribuído em duas categorias principais de concorrência baseadas nas APIs do C++17:

- **Execução de Tarefas Cíclicas (Time-Triggered)**

Implementadas via *boost::asio::steady_timer* associadas a instâncias dedicadas de *boost::asio::io_context*. Cada contexto de E/S executa de forma bloqueante através do método *io_context::run()*, encapsulado dentro de sua própria *std::thread* nativa. Isso garante o isolamento de CPU para que os determinismos de 20 ms (Odometria), 80 ms (Comando e PID) e 100 ms (LIDAR) operem concorrentemente.

- **Execução de Tarefas Reativas (Event-Triggered)**

Diferente das tarefas cíclicas, as threads da Inspeção por Câmera e do Coletor de Dados operam de forma aperiódica (sob demanda). Elas utilizam a API nativa de sincronização *std::condition_variable* associada a um *std::unique_lock*. Em vez de realizarem *busy-wait*, essas threads permanecem suspensas pelo sistema operacional no método *.wait()*, consumindo 0% de CPU. Elas são acordadas de forma determinística por meio de interrupções de software via *.notify_one()*, disparadas imediatamente pelas threads produtoras (LIDAR e Odometria) assim que um evento crítico ou um novo dado é registrado.

3 Resultados dos Testes Preliminares

Para validar o comportamento do sistema, utilizamos o arquivo *simulacao_superficie.csv* para emular as leituras do sensor LIDAR. Esse arquivo contém uma sequência de medidas simulando o teto plano do túnel a 10.0 m de altura, intercalada com variações de distância que representam anomalias geométricas (buracos e saliências).

Os testes em malha fechada demonstraram o seguinte comportamento dinâmico, em LOGS no Terminal:

O filtro de média móvel (LidarFilter) processou as leituras do arquivo a cada 100 ms, eliminando ruídos isolados e calculando a média contínua da superfície. Ao passar por uma leitura superior a 10.5 m (configurando um buraco) ou inferior a 9.5 m (uma saliência), a thread de reconstrução detectou a anomalia e disparou o alarme através da variável de condição *cv_camera*, ativando também a flag *e_inspecao*. Assim que essa flag foi acionada, o gerenciador de navegação alterou instantaneamente o setpoint de velocidade de 5.0 m/s (ou 1.0 m/s no modo automático) para 0.2 m/s. O controlador PID respondeu aplicando uma aceleração negativa estável, desacelerando o robô com segurança. Por fim, a thread da Câmera acordou sob demanda, processou a simulação pesada da IA e, após a conclusão, resetou as flags de estado, permitindo que o robô retomasse sua velocidade nominal de cruzeiro de forma totalmente autônoma, sem travamentos ou deadlocks.

4 Conclusão

Este trabalho desenvolveu a Etapa 1 de um sistema embarcado concorrente para inspeção autônoma de túneis, consolidando uma arquitetura de software capaz de atender simultaneamente a requisitos de tempo real rígido e processamento reativo orientado a eventos.

A solução implementada em C++ com **Boost.Asio** demonstrou, por meio dos testes preliminares, que as seis threads concorrentes coexistem sem deadlocks, condições de corrida ou inanição. O uso de **steady_timer** garantiu periodicidade estável nas tarefas cíclicas, enquanto o padrão **condition_variable** assegurou que as threads reativas, Câmera e Coletor, permanecessem dormientes até a ocorrência de eventos críticos, preservando os ciclos de CPU para as malhas de controle. A detecção de anomalias pelo LIDAR, o slowdown automático do robô e o acionamento sob demanda da inspeção visual operaram de forma integrada e autônoma, confirmando a viabilidade da arquitetura proposta.

A separação em buffers especializados com mutexes evitou a contenção global que um mutex único causaria e permitiu que threads de criticidades distintas acessassem a memória compartilhada com mínima interferência mútua.

4.1 Próximos Passos e Testes

O avanço para a Etapa 2 envolverá as seguintes frentes prioritárias:

- **Substituição dos Mocks pela Interface Real:** Os valores fixos de joystick e modo de operação codificados na thread de Comando de Navegação serão substituídos por dados reais publicados pelo Front-End Python via Pub/Sub, completando a integração entre simulador e sistema embarcado.

```
[ODOMETRIA]: Distancia percorrida: 6 m
[ODOMETRIA]: Distancia percorrida: 6 m
[ODOMETRIA]: Distancia percorrida: 6 m
[LIDAR] Medida atual: 9 m
[LIDAR] FALHA: SALIENCIA detectada! (Dist/Altura: 9m)
[CAMERA] Falha detectada! Iniciando processamento de IA...
[PID] Setpoint: 1 m/s | Aceleracao calculada: 3.648 m/s2
[ODOMETRIA]: Distancia percorrida: 6 m
[ODOMETRIA]: Distancia percorrida: 6 m
[CAMERA] Inspecao concluida em 40.5412 ms.
[ODOMETRIA]: Distancia percorrida: 6 m
[ODOMETRIA]: Distancia percorrida: 6 m
[PID] Setpoint: 1 m/s | Aceleracao calculada: 3.656 m/s2
[ODOMETRIA]: Distancia percorrida: 7 m
[LIDAR] Medida atual: 9.2 m
[LIDAR] FALHA: SALIENCIA detectada! (Dist/Altura: 9.2m)
[CAMERA] Falha detectada! Iniciando processamento de IA...
[ODOMETRIA]: Distancia percorrida: 7 m
[ODOMETRIA]: Distancia percorrida: 7 m
[CAMERA] Inspecao concluida em 42.6956 ms.
[ODOMETRIA]: Distancia percorrida: 7 m
[PID] Setpoint: 5 m/s | Aceleracao calculada: 10.196 m/s2
[ODOMETRIA]: Distancia percorrida: 7 m
[ODOMETRIA]: Distancia percorrida: 7 m
[LIDAR] Medida atual: 9.4 m
[LIDAR] FALHA: SALIENCIA detectada! (Dist/Altura: 9.4m)
[CAMERA] Falha detectada! Iniciando processamento de IA...
[ODOMETRIA]: Distancia percorrida: 7 m
[CAMERA] Inspecao concluida em 39.1451 ms.
[ODOMETRIA]: Distancia percorrida: 7 m
[PID] Setpoint: 5 m/s | Aceleracao calculada: 7.736 m/s2
[ODOMETRIA]: Distancia percorrida: 7 m
[ODOMETRIA]: Distancia percorrida: 7 m
[ODOMETRIA]: Distancia percorrida: 7 m
[LIDAR] Medida atual: 9.6 m
[ODOMETRIA]: Distancia percorrida: 7 m
[PID] Setpoint: 5 m/s | Aceleracao calculada: 7.776 m/s2
[ODOMETRIA]: Distancia percorrida: 7 m
```

Figura 2: LOGS de execução do Terminal.

- **Teste de Estresse da Fila Produtor-Consumidor:** Será simulada a ocorrência de múltiplas anomalias consecutivas em alta frequência para verificar o comportamento da fila_medicoes sob pressão, identificando o limiar a partir do qual o Coletor não consegue drenar os dados no ritmo de produção da Odometria.
- **Enriquecimento do Dataset de Simulação|:** O arquivo simulacao_superficie.csv será expandido com sequências mais representativas, incluindo anomalias sobrepostas, ruído gaussiano e variações geométricas graduais, com o objetivo de validar os limiares do filtro de média móvel e a sensibilidade da lógica de detecção em cenários mais próximos das condições reais de um túnel.

5 Apêndice

5.1 *main.cpp*

```
1  #include <iostream>
2  #include <thread>
3  #include <chrono>
4  #include <boost/asio.hpp>
5  #include <fstream>
6  #include <iomanip>
7  #include <string>
8  #include <functional>
9  #include <condition_variable>
10
11 // Inclusão dos Buffers e Blocos de Execução
12 #include "tasks.hpp"
13 #include "shared_buffers.hpp"
14 #include "NavigationManager.hpp"
15 #include "PIDController.hpp"
16 #include "Coletor.hpp"
17 #include "Camera.hpp"
18
19 // Thread responsável por ler os comandos (joystick/botoes) e definir o Setpoint
20 // Roda ciclicamente a cada 80ms
21 void t_comando_navegacao(NavBuffer& nav, SensorBuffer& sensor) {
22     NavigationManager nav_manager;
23
24     boost::asio::io_context io_com_nav;
25     boost::asio::steady_timer timer_com_nav(io_com_nav, boost::asio::chrono::milliseconds(80));
26
27     std::function<void(const boost::system::error_code&)> loop_comando;
28
29     loop_comando = ([&](const boost::system::error_code& erro){
30
31         if(erro) return;
32
33         // ----- Mock de entradas -----
34         bool c_automatico = false;
35         bool c_man = true;
36         bool c_para = false;
37
38         double velocidade_joystick = 5.0;
39
40         bool em_inspecao = false;
41         {
42             // Bloqueia o mutex da câmera apenas para ler a flag
```

```

43         std::lock_guard<std::mutex> lock(sensor.mtx_camera);
44         em_inspecao = sensor.e_inspecao;
45     }
46
47     // Processamento da lógica de estado da navegação
48     nav_manager.updateMode(c_automático, c_man);
49     nav_manager.processInputs(velocidade_joystick, c_para, em_inspecao);
50
51     // ----- Zona Crítica: Escrita no Buffer Compartilhado -----
52     {
53         std::lock_guard<std::mutex> lock(nav.mtx);
54         nav.e_automático = (nav_manager.getMode() == NavMode::AUTOMATIC);
55         nav.j_sp_velocidade = nav_manager.getTargetSpeed();
56     }
57
58     timer_com_nav.expires_at(timer_com_nav.expiry() + boost::asio::chrono::milliseconds(
59     timer_com_nav.async_wait(loop_comando);
60 });
61
62     timer_com_nav.async_wait(loop_comando);
63     io_com_nav.run();
64 }
65
66 // Thread responsável pelo controle PID (Sem mudanças aqui)
67 void t_controle_navegacao(NavBuffer& nav, SensorBuffer& sensor) {
68     PIDController pid(1.0, 0.1, 0.05, 0.08);
69
70     boost::asio::io_context io_PID_nav;
71     boost::asio::steady_timer timer_PID_nav(io_PID_nav, boost::asio::chrono::milliseconds(80
72
73     std::function<void(const boost::system::error_code&> loop_PID;
74
75     loop_PID = ([&](const boost::system::error_code& erro){
76         if(erro) return;
77
78         float setpoint_atual = 0.0;
79         {
80             std::lock_guard<std::mutex> lock(nav.mtx);
81             setpoint_atual = nav.j_sp_velocidade;
82         }
83
84         double velocidade_atual_robo = 0.0;
85         {
86             // Tranca a porta só para ler a velocidade com segurança
87             std::lock_guard<std::mutex> lock_leituras(sensor.mtx_leituras);
88             velocidade_atual_robo = sensor.velocidade_real_medida;
89         }

```

```

90
91     double saida_aceleracao = pid.compute(static_cast<double>(setpoint_atual), velocidad
92
93     {
94         std::lock_guard<std::mutex> lock_tela(mtx_console);
95         std::cout << "[PID] Setpoint: " << setpoint_atual << " m/s | Aceleracao calculad
96     }
97
98     {
99         std::lock_guard<std::mutex> lock(nav.mtx);
100         nav.o_aceleracao = saida_aceleracao;
101     }
102
103     timer_PID_nav.expires_at(timer_PID_nav.expiry() + boost::asio::chrono::milliseconds(
104     timer_PID_nav.async_wait(loop_PID);
105 });
106
107 timer_PID_nav.async_wait(loop_PID);
108 io_PID_nav.run();
109 }
110
111 int main() {
112     std::cout << "Iniciando Sistema de Inspecao do Tunel...\n";
113
114     NavBuffer nav;
115     SensorBuffer sensor;
116
117     std::thread th_comando(t_comando_navegacao, std::ref(nav), std::ref(sensor));
118
119     std::thread th_controle(t_controle_navegacao, std::ref(nav), std::ref(sensor));
120     std::thread th_distancia(t_calculo_distancia, std::ref(nav), std::ref(sensor));
121     std::thread th_teto(t_reconstrucao_teto, std::ref(sensor));
122     std::thread th_camera(t_inspecao_camera, std::ref(sensor));
123     std::thread th_coletor(t_coletor_dados, std::ref(sensor));
124
125     th_comando.join();
126     th_controle.join();
127     th_distancia.join();
128     th_teto.join();
129     th_camera.join();
130     th_coletor.join();
131
132     return 0;
133 }

```

5.2 shared_buffers.hpp

```
1  #ifndef SHARED_BUFFERS_HPP
2  #define SHARED_BUFFERS_HPP
3
4  #include <mutex>
5  #include <condition_variable>
6  #include <queue>
7
8  /*
9   Define os Buffers, Mutexes, Filas e Variáveis de Condição que permitem
10  a comunicação segura entre as múltiplas threads do sistema. Garante a
11  exclusão mútua nas zonas críticas, evitando condições de corrida.
12  */
13
14  inline std::mutex mtx_console;
15
16  // Buffer de Comando e Navegação
17  // Armazena os dados de troca entre os comandos do usuário e o controle PID
18  // do robo. Protegida por MUTEX para evitar Condição de Corrida.
19  struct NavBuffer {
20      double j_sp_velocidade = 0.0; // Velocidade alvo solicitada (m/s)
21      double o_aceleracao = 0.0;    // Esforço do controle calculado pelo PID
22      bool e_automatgico = false; // Estado de operação (Manual ou Auto)
23      std::mutex mtx;              // O cadeado do PID
24  };
25
26  // Armazena uma leitura única de sensor. Empacota os dados antes de enviar para
27  // a fila (queue).
28  struct Medicao {
29      long long timestamp = 0; // Tempo da leitura (ms)
30      int i_encoder = 0;       // Odometria: Posição atual no eixo X
31      float i_lidar = 0.0;     // LIDAR: Distância até o teto
32      double nivel_confianca = 0; // Qualidade da medição (0 a 100)
33  };
34
35  // Buffer dos Sensores e Câmera
36  // Armazena os dados sensoriais e sinalização de eventos críticos
37  struct SensorBuffer {
38      // ----- Fila de Dados (Produtor-Consumidor) -----
39      // A thread de Sensores "Produz" e a thread do Coletor "Consome"
40      std::queue<Medicao> fila_medicoes;
41      std::mutex mtx_fila; // Cadeado para a inserção e remoção da fila
42
43      std::mutex mtx_leituras;
44
45      // Variável de condição para o Coletor de Dados.
```



```

46     std::condition_variable cv_coletor;
47
48     // ----- Sistema de Alarme - Interrupção (Câmera) -----
49     bool e_inspecao = false; // FLAG que indica se o teto tem buracos
50     std::mutex mtx_camera;    // Cadeado para proteger a alteração da FLAG
51
52     // Variável de Condição para a Câmera
53     bool o_liga_camera = false;
54     std::condition_variable cv_camera; // 0 alarme da câmera
55
56     float ultima_leitura_lidar = 10.0;
57
58     // Variável para compartilhar com o PID
59     double velocidade_real_medida = 0.0;
60 };
61
62 #endif

```

5.3 tasks.hpp

```

1  #ifndef TASKS_HPP
2  #define TASKS_HPP
3
4  #include "shared_buffers.hpp"
5
6
7  void t_comando_navegacao(NavBuffer& nav, SensorBuffer& sensor);
8  void t_controle_navegacao(NavBuffer& nav, SensorBuffer& sensor);
9
10 // --- Threads do Bloco de Sensores ---
11 void t_calculo_distancia(NavBuffer& nav, SensorBuffer& sensor);
12 void t_reconstrucao_teto(SensorBuffer& sensor);
13
14 #endif

```

5.4 tasks.cpp

```

1  #include <iostream>
2  #include <thread>
3  #include <chrono>
4  #include <boost/asio.hpp>
5  #include <fstream>
6  #include <string>
7  #include <vector>

```

```

8  #include <functional>
9
10 #include "tasks.hpp"
11 #include "shared_buffers.hpp"
12 #include "Lidar.hpp"
13 #include "Odometria.hpp"
14
15 /* Contém as threads responsáveis por ler a odometria e mapear o teto com LIDAR
16 */
17
18 // --- Odometria: Produtor de dados de distância ---
19 void t_calculo_distancia(NavBuffer& nav, SensorBuffer& sensor) {
20     Odometria odo;
21
22     double velocidade_fisica = 0.0;
23     double posicao_fisica = 0.0;
24     int ultimo_metro_inteiro = 0;
25     int distancia_anterior = 0;
26     double dt = 0.02; // 20ms do timer
27
28     bool pulso_fisico_encoder = false;
29
30     boost::asio::io_context io_odo;
31     boost::asio::steady_timer timer_odo(io_odo, boost::asio::chrono::milliseconds(20));
32
33     std::function<void(const boost::system::error_code&)> loop_odo;
34
35     loop_odo = ([&](const boost::system::error_code& erro) {
36         if(erro) return;
37
38         // Freando o robô no mundo real - Integração
39         double aceleracao = nav.o_aceleracao * 0.1;
40
41         velocidade_fisica += aceleracao * dt;
42         if(velocidade_fisica < 0.0) velocidade_fisica = 0.0; // Impede o robô de dar ré
43
44         posicao_fisica += velocidade_fisica * dt;
45
46         // Encoder só vira quando acumula 1 metro
47         if ((int)posicao_fisica > ultimo_metro_inteiro) {
48             pulso_fisico_encoder = !pulso_fisico_encoder;
49             ultimo_metro_inteiro = (int)posicao_fisica;
50         }
51
52         int dist_x = odo.atualizar(pulso_fisico_encoder);
53
54         // Derivação para encontrar a velocidade

```

```

55     double variacao_distancia = dist_x - distancia_anterior;
56     double velocidade_medida = variacao_distancia / dt;
57
58     {
59         std::lock_guard<std::mutex> lock_tela(mtx_console);
60         std::cout << "[ODOMETRIA]: Distancia percorrida: " << dist_x << " m\n";
61     }
62
63     // Atualiza a memória para o próximo ciclo de 20ms
64     distancia_anterior = dist_x;
65
66     // Guarda a velocidade medida no buffer para a thread do PID ler
67     sensor.velocidade_real_medida = velocidade_medida;
68
69     Medicao m;
70     m.i_encoder = dist_x;
71     // Timestamp simples para o coletor
72     m.timestamp = std::chrono::steady_clock::now().time_since_epoch().count();
73     m.i_lidar = sensor.ultima_leitura_lidar;
74     m.nivel_confianca = 100; // Valor padrão inicial
75
76     {
77         // Tranca a porta para escrever a velocidade e ler o Lidar com segurança
78         std::lock_guard<std::mutex> lock_leituras(sensor.mtx_leituras);
79         sensor.velocidade_real_medida = velocidade_medida;
80         m.i_lidar = sensor.ultima_leitura_lidar;
81     }
82
83     // ----- Zona Crítica: Guardar na fila para o Coletor -----
84     {
85         std::lock_guard<std::mutex> lock_fila(sensor.mtx_fila);
86         sensor.fila_medicoes.push(m);
87     }
88
89     sensor.cv_coletor.notify_one();
90
91     timer_odo.expires_at(timer_odo.expiry() + boost::asio::chrono::milliseconds(20));
92     timer_odo.async_wait(loop_odo);
93 });
94
95 timer_odo.async_wait(loop_odo);
96 io_odo.run();
97 }
98
99 // --- LIDAR / Reconstrução do teto: Identifica falhas e aciona a câmera ---
100 void t_reconstrucao_teto(SensorBuffer& sensor) {
101     LidarFilter filtro(5);

```

```

102
103 // Inicializa o filtro com valor neutro
104 for(int i = 0; i < 5; i++) { filtro.calcular(10.0f); }
105
106 std::vector<float> valores_teste;
107 std::ifstream arquivo_teto("simulacao_superficie.csv");
108 std::string linha;
109
110 // Tenta carregar o arquivo CSV
111 if (arquivo_teto.is_open()) {
112     while (std::getline(arquivo_teto, linha)) {
113         if(linha.empty()) continue;
114         try {
115             valores_teste.push_back(std::stof(linha));
116         } catch (...) { }
117     }
118     arquivo_teto.close();
119     {
120         std::lock_guard<std::mutex> lock_tela(mtx_console);
121         std::cout << "[SISTEMA] Arquivo simulacao_superficie.csv carregado com " << valores_teste.size() << "\n";
122     }
123 }
124
125 // Proteção: Se o arquivo falhou ou está vazio, garante que o programa não dê crash
126 if (valores_teste.empty()) {
127     {
128         std::lock_guard<std::mutex> lock_tela(mtx_console);
129         std::cerr << "[AVISO] Arquivo de simulacao vazio ou nao encontrado. Usando teto\n";
130     }
131
132     valores_teste.push_back(10.0f);
133 }
134
135
136 int indice_teste = 0;
137 boost::asio::io_context io_lidar;
138 boost::asio::steady_timer timer_lidar(io_lidar, boost::asio::chrono::milliseconds(100));
139
140 std::function<void(const boost::system::error_code&)> loop_lidar;
141
142 loop_lidar = ([&](const boost::system::error_code& erro){
143     if(erro) return;
144
145     // Leitura do valor atual do vetor (circular)
146     float valor_atual = valores_teste[indice_teste];
147     indice_teste = (indice_teste + 1) % valores_teste.size();
148

```

```

149     float media = filtro.calcular(valor_atual);
150
151     // Proteção: Tranca a porta para atualizar o SensorBuffer
152     {
153         std::lock_guard<std::mutex> lock_memoria(sensor.mtx_leituras);
154         sensor.ultima_leitura_lidar = media;
155     }
156
157     // Proteção: Imprime a medida atual para o usuário ver
158     {
159         std::lock_guard<std::mutex> lock_tela(mtx_console);
160         std::cout << "[LIDAR] Medida atual: " << media << " m\n";
161     }
162
163     bool falha_detectada = false;
164     float altura_ideal = 10.0f;
165     float margem_erro = 0.5f;
166
167     // Lógica de Detecção
168     if (media > (altura_ideal + margem_erro)){
169         {
170             std::lock_guard<std::mutex> lock_tela(mtx_console);
171             std::cout << "[LIDAR] FALHA: BURACO detectado! (Dist/Altura: " << media << "m)\n";
172         }
173
174         falha_detectada = true;
175     }
176     else if(media < (altura_ideal - margem_erro)){
177         {
178             std::lock_guard<std::mutex> lock_tela(mtx_console);
179             std::cout << "[LIDAR] FALHA: SALIENCIA detectada! (Dist/Altura: " << media << "m)\n";
180         }
181
182         falha_detectada = true;
183     }
184
185     // Se achou uma falha, aciona o Alarme e sinaliza o Slowdown
186     if (falha_detectada) {
187         {
188             std::lock_guard<std::mutex> lock_camera(sensor.mtx_camera);
189             sensor.e_inspecao = true;      // Ativa o Slowdown no NavigationManager
190             sensor.o_liga_camera = true;   // Acorda a thread da Câmera
191         }
192         sensor.cv_camera.notify_one();
193     }
194
195     timer_lidar.expires_at(timer_lidar.expiry() + boost::asio::chrono::milliseconds(100));

```

```

196     timer_lidar.async_wait(loop_lidar);
197 });
198
199     timer_lidar.async_wait(loop_lidar);
200     io_lidar.run();
201 }

```

5.5 *PIDController.hpp*

```

1  #ifndef PID_CONTROLLER_HPP
2  #define PID_CONTROLLER_HPP
3
4  class PIDController {
5  private:
6      double Kp, Ki, Kd;
7      double integral;
8      double last_error;
9      double dt;
10     double output_limit;
11     public:
12     PIDController(double kp, double ki, double kd, double sample_time);
13     double compute(double setpoint, double current_value);
14     void reset();
15     void setGains(double kp, double ki, double kd);
16 };
17
18 #endif

```

5.6 *PIDController.cpp*

```

1  #include "PIDController.hpp"
2  #include <algorithm>
3
4  // Bloco: Controle de Navegação
5
6  // ----- Construtor da Classe PIDController -----
7  // integral(0.0): acumula o erro ao longo do tempo. Parcela I
8  // last_error(0.0): Parcela D.
9  // dt(sample_time): tempo de amostragem (Delta T).
10 // output_limit(100.0): PWM de 100%
11 PIDController::PIDController(double kp, double ki, double kd, double sample_time)
12     : Kp(kp), Ki(ki), Kd(kd), integral(0.0), last_error(0.0),
13     dt(sample_time), output_limit(100.0) {}
14

```

```

15 // Permite troca de parâmetros P, I e D com o robô em funcionamento.
16 void PIDController::setGains(double kp, double ki, double kd) {
17     Kp = kp;
18     Ki = ki;
19     Kd = kd;
20 }
21
22 // Troca de modo de Manual para Automático: zera tudo.
23 void PIDController::reset() {
24     integral = 0.0;
25     last_error = 0.0;
26 }
27
28 // Motor do PID
29 double PIDController::compute(double setpoint, double current_value) {
30     // Onde eu quero estar (setpoint) - onde eu estou agora.
31     double error = setpoint - current_value;
32
33     // Proporcional
34     double P = Kp * error;
35
36     // Soma do erro ao longo do tempo: Integral
37     integral += error * dt;
38     double I = Ki * integral;
39
40     // Inclinação da reta do erro: Derivativo
41     double derivative = (error - last_error) / dt;
42     double D = Kd * derivative;
43
44     // Atualiza a memória para o próximo ciclo
45     last_error = error;
46
47     double output = P + I + D;
48
49     // Robo só aceita força de -100 a 100
50     if (output > output_limit) {
51         output = output_limit;
52         integral -= error * dt;
53
54     } else if (output < -output_limit) {
55         output = -output_limit;
56         integral -= error * dt;
57     }
58
59     return output;
60 }

```

5.7 NavigationManager.hpp

```
1  #ifndef NAVIGATION_MANAGER_HPP
2  #define NAVIGATION_MANAGER_HPP
3
4  enum class NavMode { MANUAL, AUTOMATIC };
5
6  class NavigationManager {
7  private:
8      NavMode currentMode;
9      double targetSpeed;
10
11  public:
12      NavigationManager();
13
14      void updateMode(bool cmdAuto, bool cmdMan);
15
16      void processInputs(double joystickSpeed, bool btnStop, bool slowDown);
17
18      NavMode getMode() const;
19      double getTargetSpeed() const;
20  };
21
22  #endif
```

5.8 NavigationManager.cpp

```
1  #include "NavigationManager.hpp"
2
3  // Bloco: Comando de Navegação
4
5  // ----- Construtor da Classe NavigationManager -----
6  NavigationManager::NavigationManager() : currentMode(NavMode::MANUAL),
7      targetSpeed(0.0) {}
8
9  // Seleção: Manual / Automático
10 void NavigationManager::updateMode(bool cmdAuto, bool cmdMan) {
11     if (cmdAuto) {
12         currentMode = NavMode::AUTOMATIC;
13     } else if (cmdMan) {
14         currentMode = NavMode::MANUAL;
15     }
16 }
17
18 // Acelerador e Freio com lógica de redução de velocidade (slowDown)
```



```

19 void NavigationManager::processInputs(double joystickSpeed, bool btnStop, bool slowDown) {
20     // 1. Botão de Emergência: Prioridade máxima, o robô para.
21     if (btnStop) {
22         targetSpeed = 0.0;
23         return;
24     }
25
26     // 2. Lógica de Inspeção (slowDown): Se a câmera estiver processando,
27     // a velocidade cai para um valor baixo (ex: 0.2) para garantir a qualidade.
28     if (slowDown) {
29         targetSpeed = 1;
30     }
31     else {
32         // 3. Operação Normal
33         if (currentMode == NavMode::MANUAL) {
34             // Velocidade vem do controle joystick
35             targetSpeed = joystickSpeed;
36         } else {
37             // Velocidade automática nominal: 1.0
38             targetSpeed = 1.0;
39         }
40     }
41 }
42
43 NavMode NavigationManager::getMode() const {
44     return currentMode;
45 }
46
47 double NavigationManager::getTargetSpeed() const {
48     return targetSpeed;
49 }

```

5.9 Camera.hpp

```

1  #ifndef CAMERA_HPP
2  #define CAMERA_HPP
3
4  #include "shared_buffers.hpp"
5
6  // Thread que simula a inspeção visual detalhada
7  void t_inspecao_camera(SensorBuffer& sensor);
8
9  // Função para simular carga de CPU (IA)
10 void processamento_pesado_ia();
11
12 #endif

```

5.10 Camera.cpp

```
1  #include "Camera.hpp"
2  #include <iostream>
3  #include <vector>
4  #include <cmath>
5  #include <chrono>
6
7  // Função otimizada para garantir carga real de CPU
8  void processamento_pesado_ia() {
9      double resultado = 0.0;
10     for (int i = 0; i < 500000; ++i) {
11         resultado += std::sin(i) * std::cos(i) * std::tan(i);
12     }
13
14     // Impede que o compilador ignore o loop (Dead Code Elimination)
15     if(resultado == 0.00000123) std::cout << resultado;
16 }
17
18 void t_inspecao_camera(SensorBuffer& sensor) {
19     while (true) {
20         // Bloqueio inicial para esperar o sinal
21         std::unique_lock<std::mutex> lock(sensor.mtx_camera);
22
23         // Aguarda sinalização do LIDAR (o_liga_camera)
24         sensor.cv_camera.wait(lock, [&sensor] {
25             return sensor.o_liga_camera;
26         });
27
28         {
29             std::lock_guard<std::mutex> lock_tela(mtx_console);
30             std::cout << "[CAMERA] Falha detectada! Iniciando processamento de IA...\n";
31         }
32
33         // Libera o LOCK durante o processamento pesado
34         lock.unlock();
35
36         auto start = std::chrono::high_resolution_clock::now();
37         processamento_pesado_ia();
38         auto end = std::chrono::high_resolution_clock::now();
39
40         std::chrono::duration<double, std::milli> elapsed = end - start;
41
42         lock.lock();
43
44         {
45             std::lock_guard<std::mutex> lock_tela(mtx_console);
```

```

46         std::cout << "[CAMERA] Inspecao concluida em " << elapsed.count() << " ms.\n";
47     }
48
49     sensor.o_liga_camera = false;
50     sensor.e_inspecao = false;
51
52     // O lock é liberado automaticamente ao voltar para o wait ou fim do loop
53 }
54 }

```

5.11 Coletor.hpp

```

1  #ifndef COLETOR_HPP
2  #define COLETOR_HPP
3
4  #include <iomanip>
5  #include <iostream>
6  #include <fstream>
7  #include <string>
8  #include "shared_buffers.hpp"
9
10 // Classe Coletor de Dados: lógica de salvar arquivos no HD
11 class ColetorCSV {
12 private:
13     std::ofstream arquivo;
14
15 public:
16     // Construtor: Abre o arquivo assim que a classe for instanciada
17     ColetorCSV(const std::string& nome_arquivo) {
18         arquivo.open(nome_arquivo, std::ios::app);
19         if (!arquivo.is_open()) {
20             std::cerr << "[ERRO] Falha ao abrir o arquivo de log: "
21                 << nome_arquivo << std::endl;
22         }
23     }
24
25     // Destrutor: Fecha o arquivo sozinho para não corromper os dados
26     ~ColetorCSV() {
27         if (arquivo.is_open()) {
28             arquivo.close();
29         }
30     }
31
32     bool estaAberto() const {
33         return arquivo.is_open();
34     }
35 }

```

```

34     }
35
36     // Método para salvar os dados
37     void gravar(const Medicao& dado) {
38         if (arquivo.is_open()) {
39             arquivo << dado.timestamp << ","
40                 << dado.i_encoder << ","
41                 << std::fixed << std::setprecision(2) << dado.i_lidar << ","
42                 << std::fixed << std::setprecision(2)
43                 << dado.nivel_confianca << std::endl;
44         }
45     }
46 };
47
48 // Protótipo da função da thread
49 void t_coletor_dados(SensorBuffer& sensor);
50
51 #endif

```

5.12 Coletor.cpp

```

1  #include "Coletor.hpp"
2  #include <mutex>
3  #include <condition_variable>
4  #include <cmath> // Necessário para pow() e sqrt()
5
6  // Thread responsável por consumir os dados da fila, analisar a
7  // confiança e salvar no disco
8  void t_coletor_dados(SensorBuffer& sensor) {
9
10     ColetorCSV logger("telemetria_inspecao.csv");
11
12     if (!logger.estaAberto()) {
13         return;
14     }
15
16     // Variáveis para guardar a última posição e calcular a distância
17     double ultimo_x = 0.0;
18     double ultimo_y = 0.0;
19
20     while (true) {
21         std::unique_lock<std::mutex> lock_fila(sensor.mtx_fila);
22
23         sensor.cv_coletor.wait(lock_fila, [&sensor] {
24             return !sensor.fila_medicoes.empty();

```

```

25     });
26
27     while (!sensor.fila_medicoes.empty()) {
28         Medicao dado = sensor.fila_medicoes.front();
29         sensor.fila_medicoes.pop();
30
31         // --- Análise de Confiança ---
32
33         // Calcula a distância entre o ponto atual e o último ponto
34         double distancia = std::sqrt(std::pow(dado.i_encoder - ultimo_x, 2) +
35                                     std::pow(dado.i_lidar - ultimo_y, 2));
36
37         // Lógica de Confiança: Se a distância aumenta, a confiança cai.
38         // O fator "5.0" é um multiplicador de sensibilidade
39         double confianca_calculada = 100.0 - (distancia * 5.0);
40
41         // Travas de segurança para não ter confiança negativa ou acima de 100%
42         if (confianca_calculada < 0.0) confianca_calculada = 0.0;
43         if (confianca_calculada > 100.0) confianca_calculada = 100.0;
44
45         dado.nivel_confianca = confianca_calculada;
46         ultimo_x = dado.i_encoder;
47         ultimo_y = dado.i_lidar;
48
49         // Destranca para gravar no HD
50         lock_fila.unlock();
51         logger.gravar(dado);
52         lock_fila.lock();
53     }
54 }
55 }

```

5.13 Odometria.hpp

```

1  #ifndef ODOMETRIA_HPP
2  #define ODOMETRIA_HPP
3
4  class Odometria {
5  private:
6      bool estado_anterior = false;
7      int distancia_total_x = 0;
8
9  public:
10     Odometria();
11     int atualizar(bool sinal_encoder);

```

```

12     int get_distancia();
13 };
14
15 #endif

```

5.14 Odometria.cpp

```

1  #include "Odometria.hpp"
2
3  #include <iostream>
4
5  // Bloco: Cálculo da distância percorrida
6
7  // ----- Contrutor da Classe Odometria -----
8  // distancia_total_x(0): robô é ligado, roda está parada -> distância inicial = 0.
9  // estado_anterior(false): sensor começou desligado.
10 Odometria::Odometria() : estado_anterior(false), distancia_total_x(0) {}
11
12 // Borda de Subida
13 int Odometria::atualizar(bool sinal_encoder) {
14
15     // Detecta borda de subida
16     // garante contagem única apenas na exata transição de 0 para 1.
17     if (sinal_encoder && !estado_anterior) {
18         distancia_total_x++; // Soma 1 metro
19     }
20
21     // Atualiza memória
22     estado_anterior = sinal_encoder;
23
24     return distancia_total_x;
25 }
26
27 int Odometria::get_distancia() {
28     return distancia_total_x;
29 }

```

5.15 Lidar.hpp

```

1  #ifndef LIDAR_HPP
2  #define LIDAR_HPP
3
4  #include <vector>
5

```

```

6 class LidarFilter {
7 private:
8     std::vector<int> buffer;
9     int head = 0;
10    int window_size;
11    float ultima_media = 0.0;
12
13 public:
14     // Escolha: size = 5. Com a janela cíclica de 100ms, uma janela de tamanho 5
15     // dá um atraso de meio segundo, filtrando picos isolados do sensor.
16     LidarFilter(int size = 5);
17     float calcular(float nova_leitura);
18     float get_ultima_media();
19 };
20
21 #endif

```

5.16 Lidar.cpp

```

1  #include "Lidar.hpp"
2  #include <numeric>
3  #include <iostream>
4
5  // Bloco: Reconstrução superfície do teto do túnel
6
7  // Lógica: Filtro de Média Móvel com Buffer Circular
8  // Guarda as última "N" leituras e tira uma média, suavizando a visão
9  // do robô e eliminando ruídos.
10
11 // ----- Construtor da classe LidarFilter -----
12 // size: tamanho da janela.
13 // buffer(size, 0): cria uma lista com 'size' espaços.
14 // window_size(size): guarda o número 'size' para calcular a média.
15 LidarFilter::LidarFilter(int size) : buffer(size, 0), window_size(size) {}
16
17 // Buffer Circular
18 float LidarFilter::calcular(float nova_leitura) {
19
20     // Substitui a leitura mais velha pela mais nova.
21     buffer[head] = nova_leitura;
22
23     // Lógica do buffer circular: quando o resto da divisão da
24     // posição pelo tamanho da janela for 0
25     // (indicando estar na última posição do buffer),
26     // head volta para a primeira posição.

```

```

27     head = (head + 1) % window_size;
28
29     // Função accumulate soma todos os itens da lista.
30     float soma = std::accumulate(buffer.begin(), buffer.end(), 0.0);
31
32     ultima_medida = soma / window_size;
33
34     return ultima_medida;
35 }
36
37 // Para leitura da última medida
38 float LidarFilter::get_ultima_medida() {
39     return ultima_medida;
40 }

```